Seminar in Software Engineering

HW1 plus Workshop – Zero Analysis with an Extended Lattice

READ THE ENTIRE ASSIGNMENT BEFORE STARTING.

Deadline: October 27, 2026 **(15:59) (no extensions)**

Introduction

In HW1 you implemented a zero analysis over the lattice $Sign = \{\bot, -, 0, +, \top\}$. That lattice loses information as soon as two different signs meet. Consider:

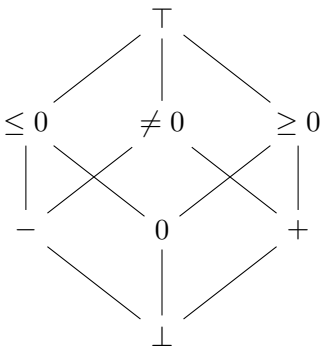
```
int x = 1;
if (m > 0) {
    x = -1;
}
int j = n / x;    // HW1 reports it: + merged with - is TOP
```

The variable `x` is never zero, but $+ \sqcup - = \top$, and $\top$ may be zero, so the HW1 analysis reports a *false alarm*. The same happens with a loop counter that starts at 0 and is incremented: $0 \sqcup + = \top$.

In this homework you will extend the lattice with three new values that keep that information:

- ≤ 0 (non-positive): the value is negative or zero.
- ≥ 0 (non-negative): the value is positive or zero.
- $\neq 0$ (not zero): the value is negative or positive.

The new lattice has eight elements:



Each element stands for a set of possible signs: $\bot$ for none, $-$, 0 and $+$ for one, ≤ 0, $\neq 0$ and ≥ 0 for two, and $\top$ for all three. The order is the inclusion of those sets, so $a \sqcup b$ is the smallest element that includes the signs of both a and b: for example, $+ \sqcup -$ is $\neq 0$ and $0 \sqcup +$ is ≥ 0.

The analysis is otherwise the same as in HW1: a *may forward dataflow analysis* with

$$IN[n] = \bigsqcup_{n' \in pred(n)} OUT[n'] \qquad OUT[n] = Transfer[n, IN[n]]$$

and a division `x = y / z` is reported as a possible division by zero when $IN[n](z)$ may be zero (now: when it is 0, ≤ 0, ≥ 0 or $\top$).

The transfer functions must be as precise as possible: the result of an operation is the smallest element of the lattice that includes every possible result. Ignore overflow (the operations are those of the integers), remember that integer division truncates ($1/2 = 0$, $-1/2 = 0$), and that a division by zero has no result: `y / z` only considers the values of `z` that are not zero.

Part 1: Defining the extended Zero Analysis

The exercises in Part 1 are a warm-up before starting the implementation. They are not submitted, but they serve as a basis for discussing any questions about the semantics of the analysis.

Exercise 1

Method ZeroAbstractValue::merge

Complete the table of $a \sqcup b$.

$a \sqcup b$	$\bot$	$-$	0	$+$	≤ 0	≥ 0	$\neq 0$	$\top$
$\bot$								
$-$								
0								
$+$								
≤ 0								
≥ 0								
$\neq 0$								
$\top$								

Exercise 2

Methods ZeroAbstractValue::add, substract, multiplyBy and divideBy

Complete the following tables to characterize the transfer function ($Transfer$) of the statements `x = y + z`, `x = y - z`, `x = y * z` and `x = y / z` (an **integer** division). Each row is the value of $IN[n](y)$ and each column the value of $IN[n](z)$; each cell is the value of $OUT[n](x)$.

Hint: You don't need to reason about 64 cases per table. Think of each value as its set of signs: the result of $a \circ b$ includes every sign that $s_1 \circ s_2$ can produce, for a sign s_1 of a and a sign s_2 of b. For example, $\geq 0 + -$ includes the signs of $0 + -$ and of $+ + -$, that is, $\{-\} \cup \{-, 0, +\}$: $\top$.

$y + z$	$\bot$	$-$	0	$+$	≤ 0	≥ 0	$\neq 0$	$\top$
$\bot$								
$-$								
0								
$+$								
≤ 0								
≥ 0								
$\neq 0$								
$\top$								

$y - z$	$\bot$	$-$	0	$+$	≤ 0	≥ 0	$\neq 0$	$\top$
$\bot$								
$-$								
0								
$+$								
≤ 0								
≥ 0								
$\neq 0$								
$\top$								

$y \times z$	$\bot$	$-$	0	$+$	≤ 0	≥ 0	$\neq 0$	$\top$
$\bot$								
$-$								
0								
$+$								
≤ 0								
≥ 0								
$\neq 0$								
$\top$								

y/z	$\bot$	$-$	0	$+$	≤ 0	≥ 0	$\neq 0$	$\top$
$\bot$								
$-$								
0								
$+$								
≤ 0								
≥ 0								
$\neq 0$								
$\top$								

Exercise 3

Method ZeroValueVisitor::maybeZeroOperand

For which values of $IN[n](z)$ must the statement `x = y / z` be reported as a possible division by zero? Why is it a *may* analysis: what would it mean to report a division only when $IN[n](z) = 0$?

Exercise 4

Consider the following program:

```
public static int func(int n) {
    int i = 0;
    while (i < n) {
        i = i + 1;
    }
    int k = i + 1;
    int j = n / k;
    int r = n / i;
    return j + r;
}
```

1. Draw its control-flow graph and run the chaotic iteration algorithm with the HW1 lattice until stable values of IN and OUT are obtained for `i` and `k`. Which divisions are reported?
2. Repeat it with the extended lattice. Which divisions are reported now? Is each report a real possible division by zero?

Part 2: Implementation

You work on the same framework as in HW1 (Tai-e: `./gradlew test`, `./gradlew zeroAnalysis -PtargetClass=<targetClass>`; see `STUDENTS.md` in the repo). The repo for this homework has the same structure as the one for HW1, with two differences:

- The `enum ZeroAbstractValue` has the eight values of the extended lattice: `BOTTOM`, `NEGATIVE`, `ZERO`, `POSITIVE`, `NON_POSITIVE` (≤ 0), `NON_NEGATIVE` (≥ 0), `NOT_ZERO` ($\neq 0$) and `TOP`.
- There are new targets (`ZeroAnalysis8` to `ZeroAnalysis12`) and their tests, and the expected values of the old targets were updated where the extended lattice is more precise.

You must complete the same methods as in HW1. **Start by copying your HW1 implementation** of `ZeroValueVisitor`, `ZeroAbstractState` and `DivisionByZeroAnalysis`, and then:

- implement the methods of `ZeroAbstractValue` (`add`, `substract`, `multiplyBy`, `divideBy` and `merge`) for the eight values;
- revise `ZeroValueVisitor::maybeZeroOperand` (and anything else in your HW1 code that assumes five values).

The tests included in the project check the abstract values of the variables at certain program points of the `targets` and the divisions reported. The course also grades your submission with additional tests, which check every operation of `ZeroAbstractValue` on every pair of values against the most precise result.

Submission

A complete implementation of the Zero Analysis with the extended lattice. The tests provided by the teaching staff must pass. You may add your own if you wish.